

SIGIC

Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia

Colônia Aurora Siger | Fase 4 - Energia para Sobreviver

1. Introdução e Objetivo

Campo	Informação
Atividade	Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia (SIGIC)
Fase	Fase 4 - Energia para sobreviver
Capítulo	Cap. 1 - A Aurora Sustenta a Energia que Mantém a Colônia Viva
Linguagem	Python 3 (somente biblioteca padrão)
Integrante 1	Isabelle Caroline de Camargo Francisco / 572096
Integrante 2	Matheus Lyncoln Souza Dias / 570765
Integrante 3	Mirela Aparecida Bispo Miguel / 570830
Integrante 4	Rodrigo Abrantes Mizerani / 571808

Este documento complementa o código-fonte do SIGIC, o Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia marciana Aurora Siger. O objetivo do sistema é representar computacionalmente a infraestrutura da base e otimizar o funcionamento da sua rede energética e operacional, apoiando a tomada de decisão em situações normais e de emergência.

A solução foi construída exclusivamente com os conteúdos estudados até esta fase: grafos e algoritmos de redes (BFS, DFS e Dijkstra), matrizes e listas de adjacência, estruturas de dados em Python (listas, matrizes, tuplas e dicionários), modelagem matemática com cálculo diferencial, otimização computacional e os conceitos de armazenamento e distribuição de energia, redes inteligentes (smart grids) e governança ESG.

O sistema é executado no terminal por meio de um menu interativo. Todo o código usa apenas a biblioteca padrão do Python, sem qualquer dependência externa, conforme as regras técnicas da atividade.

Conteúdo deste documento

- Descrição da infraestrutura da colônia e dos módulos;
- Representação da rede como um grafo (vértices, arestas e pesos);

- Estruturas de dados utilizadas e sua justificativa;
- Algoritmos implementados (BFS, DFS, Dijkstra e análise de criticidade);
- Modelagem matemática e otimização;
- Análise de sustentabilidade e governança (ESG);
- Funcionalidades, instruções de execução e roteiro do vídeo.

2. Descrição da Infraestrutura da Colônia

A colônia é composta por 8 módulos funcionais. Cada módulo possui atributos operacionais relevantes para a gestão da base: consumo energético (kW), prioridade operacional (de 1, baixa, a 5, vital), capacidade de armazenamento de energia (kWh), necessidade de comunicação, status operacional e a posição no mapa da base (usada no diagrama da rede).

Código	Nome	Cons.(kW)	Prioridade	Cap.(kWh)	Comunic.	Status
MOD-EN-001	Geradores + Painéis Solares	15	5 - Vital	2000	Média	ativo
MOD-SV-002	Sistemas de Suporte à Vida	40	5 - Vital	60	Alta	ativo
MOD-HB-003	Módulos Infláveis BEAM (Habitação)	120	5 - Vital	50	Alta	ativo
MOD-LG-006	Logística e Suprimentos	60	5 - Vital	80	Alta	ativo
MOD-SC-004	Sensores Científicos	90	4 - Alta	100	Média	ativo
MOD-MN-005	Mineração e Produção ISRU	50	4 - Alta	40	Alta	ativo
MOD-AG-007	Agricultura	50	4 - Alta	40	Alta	ativo
MOD-CM-008	Comunicação	50	4 - Alta	40	Alta	ativo

Justificativa dos atributos

O consumo energético é a base do balanço de carga da rede. A prioridade operacional orienta quais módulos devem ser preservados primeiro em uma falha de geração - por isso os sistemas de suporte à vida recebem prioridade 5. A capacidade de armazenamento indica a autonomia local de cada módulo; o módulo de Geradores + Painéis Solares concentra a maior reserva (2.000 kWh). O status permite sinalizar módulos em manutenção ou alerta.

3. Representação da Rede Utilizando Grafos

A infraestrutura é modelada como um grafo não direcionado e ponderado. Cada módulo é um vértice e cada conexão física entre dois módulos é uma aresta. O peso de cada aresta representa a distância (em metros) do cabeamento entre os módulos. Essa escolha é conveniente porque a perda de energia na transmissão cresce com o comprimento do cabo - assim, a rota mais curta calculada pelo Dijkstra é também a de menor desperdício energético.

Conexões da rede (arestas e pesos)

Origem	Destino	Peso (m)	Observação
MOD-EN-001	MOD-SV-002	75	Conexão comum
MOD-SV-002	MOD-HB-003	80	Conexão comum
MOD-HB-003	MOD-MN-005	80	Conexão comum
MOD-SC-004	MOD-AG-007	95	Conexão comum
MOD-AG-007	MOD-MN-005	100	Conexão comum
MOD-AG-007	MOD-SV-002	105	Conexão comum
MOD-LG-006	MOD-SC-004	75	Conexão comum
MOD-LG-006	MOD-AG-007	90	Conexão comum
MOD-CM-008	MOD-AG-007	110	PONTE CRÍTICA
MOD-EN-001	MOD-AG-007	120	Conexão comum

Matriz de adjacência (distância em metros; 0 = sem conexão)

Módulo	EN-001	SV-002	HB-003	SC-004	MN-005	LG-006	AG-007	CM-008
EN-001	0	75	0	0	0	0	120	0
SV-002	75	0	80	0	0	0	105	0
HB-003	0	80	0	0	80	0	0	0
SC-004	0	0	0	0	0	75	95	0
MN-005	0	0	80	0	0	0	100	0
LG-006	0	0	0	75	0	0	90	0
AG-007	120	105	0	95	100	90	0	110
CM-008	0	0	0	0	0	0	110	0

Justificativa da topologia

A topologia foi organizada em dois setores. O núcleo reúne os módulos vitais (Geradores, Suporte à Vida, Habitação) formando uma cadeia linear - o que, embora economize cabeamento, cria uma cadeia inicial, mas a nova topologia distribuiu melhor as conexões, resultando em apenas 1 ponte crítica detectada automaticamente pelo sistema. O setor externo (Sensores, Logística, Mineração, Agricultura e Comunicação) possui múltiplos caminhos alternativos entre seus módulos, tornando-o mais robusto a falhas individuais.

4. Grafos e Representação de Redes - Fundamentação Teórica

Esta seção apresenta os fundamentos teóricos de grafos que embasam o SIGIC, contextualizando cada conceito com a sua aplicação direta na infraestrutura da Colônia Aurora Siger.

4.1 O que são Grafos?

Um grafo é uma abstração matemática que modela relações entre elementos de um conjunto. Formalmente, um grafo G é definido por um par (V, E) , onde V é o conjunto de vértices (entidades) e E é o conjunto de arestas (relações entre pares de vértices). Uma aresta nunca liga mais do que dois vértices simultaneamente.

Na prática, grafos são usados para modelar todo tipo de rede: cidades e estradas, pessoas e relações sociais, roteadores e enlaces de comunicação. O que importa numa representação gráfica não é a disposição geométrica dos elementos, mas sim a conectividade - qual vértice está ligado a qual.

Conceitos Fundamentais

- **Vértice:** representa uma entidade (cidade, pessoa, computador, módulo de uma base).
- **Aresta:** representa uma conexão entre dois vértices (estrada, amizade, cabo de energia).
- **Adjacência:** dois vértices são adjacentes quando existe uma aresta ligando-os diretamente.
- **Grau de um vértice:** quantidade de arestas incidentes nele. Vértices com grau elevado são hubs da rede.

4.2 Tipos de Grafos

Não Direcionado vs. Direcionado

No grafo não direcionado, as arestas não possuem direção - a conexão vale nos dois sentidos. Se o vértice A está conectado ao vértice B , então B também está conectado a A (relação simétrica). No grafo direcionado (dígrafo), as arestas possuem sentido: existe $A \rightarrow B$, mas não necessariamente $B \rightarrow A$.

Ponderado vs. Não Ponderado

No grafo ponderado, cada aresta possui um número associado chamado peso, que pode representar distância, tempo, custo ou consumo de energia. No grafo não ponderado, as

arestas apenas indicam se existe ou não uma ligação - o menor caminho é o que possui menos arestas, pois todas têm o mesmo custo (1 passo).

4.3 Grafo Não Direcionado e Ponderado

Um grafo não direcionado e ponderado combina as duas características: as conexões valem nos dois sentidos e cada conexão possui um valor (peso). É o modelo mais adequado para representar redes físicas onde o fluxo pode ocorrer em qualquer direção e o custo de transporte varia de trecho em trecho.

4.4 Por que a Colônia se Encaixa nesse Modelo?

A infraestrutura da Colônia Aurora Siger foi modelada como um grafo não direcionado e ponderado por três razões fundamentais:

- **Bidirecionalidade:** os cabos de energia e comunicação entre os módulos permitem fluxo nos dois sentidos. Energia vai do Gerador ao Suporte à Vida, e sinais de monitoramento percorrem o mesmo caminho de volta.
- **Pesos significativos:** a distância física (em metros) é determinante para a perda energética - quanto mais longo o cabo, maior o desperdício (Lei de Joule). Isso permite que o Dijkstra calcule a rota de menor perda.
- **Variedade de conexões:** as distâncias variam de 75 m a 120 m, tornando um grafo não ponderado inadequado - sem pesos, todas as rotas pareceriam equivalentes e a otimização seria impossível.

Resumo: cada módulo da base é um vértice; cada cabo físico é uma aresta; e o comprimento desse cabo (em metros) é o peso da aresta.
 $G = (V, E)$ onde $V = 8$ módulos e $E = 10$ conexões físicas ponderadas.

4.5 Lista de Adjacência vs. Matriz de Adjacência

Um grafo pode ser armazenado de duas formas principais na memória. O SIGIC utiliza as duas simultaneamente, cada uma para um propósito diferente.

Lista de Adjacência

Implementada como dicionário de dicionários em Python: `grafo[u][v] = peso`. Percorrer os vizinhos de um nó é direto e eficiente - os algoritmos BFS e DFS usam exclusivamente essa estrutura.

```
# grafo[u][v] = peso da conexao
grafo = {
    'MOD-EN-001': {'MOD-SV-002': 75, 'MOD-AG-007': 120},
    'MOD-SV-002': {'MOD-EN-001': 75, 'MOD-HB-003': 80, 'MOD-AG-007': 105},
    'MOD-HB-003': {'MOD-SV-002': 80, 'MOD-MN-005': 80},
    'MOD-SC-004': {'MOD-AG-007': 95, 'MOD-LG-006': 75},
    'MOD-MN-005': {'MOD-HB-003': 80, 'MOD-AG-007': 100},
```

```
'MOD-LG-006': {'MOD-SC-004': 75, 'MOD-AG-007': 90},
'MOD-AG-007': {'MOD-EN-001': 120, 'MOD-SV-002': 105, 'MOD-SC-004': 95,
               'MOD-MN-005': 100, 'MOD-LG-006': 90, 'MOD-CM-008': 110},
'MOD-CM-008': {'MOD-AG-007': 110},
}

peso = grafo['MOD-EN-001']['MOD-SV-002'] # -> 75
```

- Eficiência $O(\text{grau})$ para percorrer vizinhos - percorre só os vizinhos, não toda a linha.
- Economia de memória $O(V+E)$ - ideal para redes esparsas como a da colônia (densidade 0,36).
- Acesso $O(1)$ por dicionário Python.

Matriz de Adjacência

Implementada como lista de listas: $M[i][j]$ = peso (0 se sem conexão). Oferece visão tabular completa da rede e consulta direta por índice.

```
# M[i][j] = distância em metros (0 = sem conexão)
      EN-001  SV-002  HB-003  SC-004  MN-005  LG-006  AG-007  CM-008
MOD-EN-001 [ 0,    75,    0,    0,    0,    0,    120,    0 ]
MOD-SV-002 [ 75,    0,    80,    0,    0,    0,    105,    0 ]
MOD-HB-003 [ 0,    80,    0,    0,    80,    0,    0,    0 ]
MOD-SC-004 [ 0,    0,    0,    0,    0,    75,    95,    0 ]
MOD-MN-005 [ 0,    0,    80,    0,    0,    0,    100,    0 ]
MOD-LG-006 [ 0,    0,    0,    75,    0,    0,    90,    0 ]
MOD-AG-007 [ 120,  105,    0,    95,  100,    90,    0,  110 ]
MOD-CM-008 [ 0,    0,    0,    0,    0,    0,  110,    0 ]
```

- Visualização tabular imediata de todas as conexões da rede.
- Consulta direta $O(1)$ por índice $M[i][j]$.
- Facilita identificar módulos isolados (linha/coluna inteira de zeros).

Decisão de projeto: o SIGIC mantém as duas representações simultaneamente. A lista de adjacência é usada pelos algoritmos (BFS, DFS, Dijkstra) por ser mais eficiente para percorrer vizinhos. A matriz é gerada sob demanda para exibição ao usuário.

4.6 Justificativa da Topologia

A rede foi organizada em torno de um hub central, equilibrando robustez, economia de cabeamento e facilidade de expansão.

- **Hub central (MOD-AG-007):** com 6 conexões, é o ponto central da rede e o único ponto de articulação. Todos os módulos se comunicam por ele ou têm caminhos que passam por ele.

- **Gerador com redundância (MOD-EN-001):** possui 2 rotas para a rede — via MOD-SV-002 (75m) e diretamente para MOD-AG-007 (120m) — eliminando o risco de isolamento total do gerador.
- **Única ponte crítica:** MOD-CM-008 ↔ MOD-AG-007 (110m). O módulo de Comunicação tem apenas 1 conexão, tornando esse cabo o único ponto crítico da rede. Recomenda-se adicionar um cabo reserva.
- **Setor distribuído (MOD-SC-004, MOD-LG-006, MOD-MN-005, MOD-HB-003):** todos conectados ao hub com caminhos alternativos, tornando a rede mais resiliente que a topologia anterior.

5. Estruturas de Dados em Python

O sistema combina quatro estruturas de dados nativas da linguagem Python, cada uma escolhida de forma estratégica com base na sua complexidade computacional, segurança de memória e adequação semântica ao modelo da Colônia Aurora Siger.

5.1 Dicionário (dict)

O dicionário em Python é implementado por meio de uma tabela de dispersão (hash table), o que garante acesso médio com complexidade $O(1)$ a qualquer elemento. No SIGIC, essa estrutura cumpre dois papéis fundamentais:

- **Mapeamento de módulos:** a chave é o código do módulo (ex.: "MOD-EN-001") e o valor é um dicionário interno com os 7 atributos operacionais. Em uma colônia em expansão com centenas de módulos, o sistema localiza qualquer dado instantaneamente, evitando a lentidão de uma varredura linear $O(n)$ que ocorreria com listas.
- **Representação do grafo (Dict-de-Dict):** a rede de distribuição é estruturada como grafo[origem][destino] = peso. Como a infraestrutura da colônia é uma rede esparsa (densidade de 0,357), essa abordagem economiza memória significativamente - o Python armazena apenas as conexões existentes, ao contrário de uma matriz estática que ocuparia espaço para todas as ligações, inclusive as inexistentes.

5.2 Tupla (tuple) - O Cofre Imutável

As tuplas representam coleções sequenciais imutáveis. No SIGIC, essa propriedade de bloqueio é essencial para garantir a consistência física do sistema:

- **Coordenadas espaciais (pos = (x, y)):** a localização geográfica de cada módulo no mapa da base é armazenada como uma tupla bidimensional. Como as instalações são estruturas físicas fixas, a imutabilidade garante que nenhuma simulação operacional ou erro em runtime altere acidentalmente a posição dos módulos no mapa.
- **Atomicidade de arestas (u, v, peso):** ao ler as conexões, o sistema agrupa origem, destino e distância em um bloco único e indivisível. Por serem imutáveis, essas tuplas são hasheáveis, permitindo seu uso seguro para rastrear caminhos e nós já visitados sem risco de mutação de estado.

5.3 Lista (list) - A Linha de Fluxo Dinâmica

A lista é um array dinâmico, ideal para situações em que a ordem dos elementos importa e o volume de dados varia constantemente. O SIGIC a utiliza em dois contextos principais:

- **Controle algorítmico:** armazena os caminhos mínimos calculados pelo Dijkstra, o histórico cronológico de nós visitados e as rotas elétricas de redundância.
- **Suporte às buscas BFS e DFS:** a lista serve de base para gerenciar a ordem de execução dos algoritmos. Permite simular pilhas para a Busca em Profundidade (usando `.append()` e `.pop()` com custo amortizado $O(1)$) e filas para a Busca em Largura, respeitando a sequência de expansão por níveis da rede.

5.4 Matriz (Lista de Listas) - A Grade de Visualização

A função `construir_matriz` projeta uma tabela bidimensional cruzando todos os módulos (linhas e colunas) para representar a adjacência da rede. Essa estrutura é gerada sob demanda, exclusivamente para facilitar a leitura humana no terminal - inspecionar uma grade visual com os pesos é muito mais intuitivo do que decifrar dicionários aninhados.

A alocação é feita com a sintaxe `matriz = [[0] * n for _ in range(n)]`. O uso de List Comprehension evita o bug crítico de aliasing da sintaxe `[[0] * n] * n`, que criaria referências duplicadas ao mesmo objeto na memória - situação em que a alteração de uma única célula corromperia a tabela inteira.

Decisão de projeto: o SIGIC mantém as duas representações simultaneamente. O dicionário de dicionários é usado pelos algoritmos (BFS, DFS, Dijkstra) por ser mais eficiente para percorrer vizinhos. A matriz é gerada sob demanda apenas para exibição ao usuário.

6. Algoritmos Implementados

6.1 BFS - Busca em Largura

Usa uma FILA (lista + ponteiro de leitura, evitando `pop(0)`). Explora o grafo por NÍVEIS a partir da origem: visita todos os vizinhos diretos antes de avançar para os vizinhos dos vizinhos. Guarda `nivel[v]` (saltos até a origem) e `anterior[v]`. Complexidade $O(V+E)$: cada vértice é enfileirado uma vez e cada aresta é examinada uma vez.

É utilizado a partir da opção 4 do menu.

>> SIGIC — Opção 4: BFS (Busca em Largura)

Origem: MOD-EN-001

Ordem de visita:

MOD-EN-001 → MOD-AG-007 → MOD-SV-002 → MOD-CM-008 → MOD-LG-006 → MOD-MN-005 → MOD-SC-004 → MOD-HB-003

Níveis (camadas):

N0 { MOD-EN-001 }

```
N1 { MOD-AG-007, MOD-SV-002 }  
N2 { MOD-CM-008, MOD-LG-006, MOD-MN-005, MOD-SC-004, MOD-HB-003 }
```

Interpretação: todos os vizinhos diretos de MOD-EN-001 são visitados antes de avançar para os vizinhos dos vizinhos (expansão por níveis).

Observe que o processamento respeita rigorosamente a hierarquia das camadas: todos os elementos do primeiro nível são explorados integralmente antes de se prosseguir ao segundo nível, garantindo a execução característica da estrutura de FILA.

6.2 DFS - Busca em Profundidade

Usa uma PILHA (append/pop no mesmo extremo). Caminha o mais fundo possível por um ramo antes de retroceder (backtrack). Diferença em relação ao BFS: o BFS espalha a busca nível a nível ("o que está próximo"); o DFS mergulha em um caminho até o fim antes de explorar outro.

Do ponto de vista da eficiência computacional e da estrutura de dados, o algoritmo DFS é extremamente eficaz para problemas de detecção de ciclos e ordenação topológica. Ao contrário do BFS, que consome memória proporcional à largura do grafo (frequentemente maior), o DFS utiliza uma estrutura de pilha que consome memória proporcional à profundidade máxima do caminho explorado, tornando-o, em muitos cenários de redes com grafos esparsos, uma alternativa mais econômica em termos de alocação de memória durante a exploração de ramos individuais da infraestrutura da colônia.

>> SIGIC — Opção 4: DFS (Busca em Profundidade)

Origem: MOD-EN-001

Ordem de visita:

MOD-EN-001 → MOD-AG-007 → MOD-CM-008 → MOD-LG-006 → MOD-SC-004 → MOD-MN-005 →
MOD-HB-003 → MOD-SV-002

Interpretação: o DFS mergulha no ramo mais profundo antes de retroceder (backtrack), usando uma PILHA em vez de FILA. Por isso a ordem de visita difere radicalmente do BFS.

Ao comparar com a lógica do BFS, nota-se que o DFS atinge o módulo MOD-AG-007 na 2ª posição, pois os vizinhos de MOD-EN-001 são explorados em ordem lexicográfica e MOD-AG-007 precede MOD-SV-002 nessa ordenação. O BFS também o acessa na primeira camada (N1), mas a ordem dos demais módulos difere radicalmente — evidenciando como a substituição da estrutura de FILA por uma PILHA altera a sequência de exploração sob a mesma topologia de rede.

6.3 Dijkstra - Caminho Mínimo

O algoritmo determina o custo mínimo acumulado partindo do ponto de origem até os demais componentes. Em cada ciclo, seleciona-se o nó não explorado que apresente a menor métrica de distância e executa-se o relaxamento de arestas: caso a soma entre $\text{dist}[\text{atual}]$ e o peso da aresta seja inferior ao registro prévio do vizinho, os campos $\text{dist}[\text{vizinho}]$ e $\text{anterior}[\text{vizinho}]$ são atualizados. A estrutura de predecessores viabiliza a recuperação integral do trajeto percorrido. Devido à escala reduzida da rede, a implementação adota busca linear com complexidade $O(V^2)$. Definir a "rota ótima" significa priorizar o menor peso acumulado, o que, na prática da colônia, traduz-se na trajetória de eficiência máxima com o menor desperdício de energia, independentemente do volume de saltos intermediários.

Perceba que o trajeto selecionado percorre **2 segmentos distintos**, priorizando a métrica de extensão total em detrimento do volume mínimo de conexões. Esse comportamento é assegurado pelo processo de relaxamento de arestas: ao finalizar a exploração de um módulo, o algoritmo consolida o valor de custo mínimo como definitivo.

6.4 Conexões Críticas e Pontos de Articulação

Para avaliar a robustez da rede, o sistema detecta pontes (conexões cuja remoção parte a rede) e pontos de articulação (módulos cuja falha isola outras partes). A estratégia é direta e didática: remove-se cada aresta (ou vértice) e verifica-se, via BFS, se o número de componentes conexos aumentou.

Conexão	Resultado
MOD-EN-001 ↔ MOD-SV-002 (75m)	Conexão comum
MOD-SV-002 ↔ MOD-HB-003 (80m)	Conexão comum
MOD-HB-003 ↔ MOD-MN-005 (80m)	Conexão comum
MOD-SC-004 ↔ MOD-AG-007 (95m)	Conexão comum
MOD-AG-007 ↔ MOD-MN-005 (100m)	Conexão comum
MOD-AG-007 ↔ MOD-SV-002 (105m)	Conexão comum
MOD-LG-006 ↔ MOD-SC-004 (75m)	Conexão comum
MOD-LG-006 ↔ MOD-AG-007 (90m)	Conexão comum
MOD-CM-008 ↔ MOD-AG-007 (110m)	PONTE CRÍTICA
MOD-EN-001 ↔ MOD-AG-007 (120m)	Conexão comum

Módulo	Resultado
MOD-AG-007 — Agricultura	Ponto de Articulação

6.5 Análise de Eficiência Operacional

- Grau: nº de conexões diretas de cada módulo (média = grau médio; maior grau = hub).
- Densidade: $(2 \times \text{arestas}) / (n \times (n-1))$ - fração das conexões possíveis que existem.
- Diâmetro: maior distância mínima (Dijkstra) entre qualquer par de módulos.
- Caminho médio: média de todas as distâncias mínimas entre pares de módulos.

Métrica	Valor
Módulos (vértices)	8
Conexões (arestas)	10
Grau médio	2,50 conexões/módulo
Módulo hub	MOD-AG-007 (6 conexões)
Densidade da rede	0,357
Caminho médio	169,5 m
Diâmetro (maior rota)	290 m (MOD-HB-003 → MOD-CM-008)
Pontes	1
Pontos de articulação	1 (MOD-AG-007)
Consumo total	475 kW

7. Modelagem Matemática e Otimização

Fenômeno modelado: o crescimento do consumo energético da colônia ao longo do tempo, à medida que novos módulos são ativados. Adotou-se o modelo logístico, porque o crescimento é rápido no início e desacelera ao se aproximar do limite de geração sustentável da base (capacidade K).

Fórmula utilizada

$$E(t) = \frac{K}{1 + A \cdot e^{(-r \cdot t)}} \quad \text{onde: } A = (K - E_0) / E_0$$

Derivada (taxa de variação da demanda):
 $\frac{dE}{dt} = r \cdot E(t) \cdot (1 - E(t)/K)$

Explicação das variáveis

- **E(t)**: demanda de energia no instante t, em kW.
- **K**: capacidade máxima de geração sustentável da base. Adotado K = 1.200 kW.
- **E0**: demanda inicial da colônia. Adotado E0 = 120 kW.
- **r**: taxa de crescimento da demanda. Adotado r = 0,15 por mês marciano.
- **A**: constante determinada pela condição inicial: $A = (K - E_0)/E_0 = 9,00$.
- **t**: tempo, em meses marcianos.

Aplicação do Cálculo Diferencial

Ponto de inflexão (máxima velocidade de crescimento):

$$t^* = \ln(A) / r = \ln(9) / 0,15 \approx 14,6 \text{ meses}$$

$$E(t^*) = K/2 = 600 \text{ kW}$$

Taxa máxima de crescimento:

$$\frac{dE}{dt}|_{\max} = r \cdot K/4 = 0,15 \times 1200 / 4 = 45,0 \text{ kW/mês}$$

Tempo para atingir 90% de K (1.080 kW): $\approx 29,3$ meses

Análise Qualitativa

A função tem o formato de "S" (sigmoide): cresce de forma quase exponencial no início, atinge a máxima velocidade no instante t^* e depois desacelera, saturando próximo de K. O instante t^* funciona como um alerta de planejamento - a partir dele, a rede entra na faixa de maior pressão sobre a geração.

Relação com o Funcionamento da Colônia

O consumo total atual da colônia é de 475 kW, cerca de 40% da capacidade K. O modelo indica que a geração deve ser ampliada antes de t^* para evitar apagões. Ações de eficiência reduzem E_0 e r , adiando a saturação e prolongando a vida útil da infraestrutura atual.

Otimização Computacional

- **Rota ótima:** o Dijkstra minimiza a distância total - e, portanto, a perda de energia - na distribuição entre módulos.
- **Priorização em blecaute:** quando a geração falha e a energia é limitada, o sistema testa todas as combinações de módulos (busca exaustiva) e escolhe, por critério de prioridade, o conjunto que mantém os sistemas mais críticos ligados.

8. Sustentabilidade e Governança (ESG)

Uso Sustentável de Energia

Roteando a distribuição pelos caminhos mais curtos (Dijkstra), o sistema minimiza as perdas de transmissão. A simulação de distribuição quantifica a energia entregue e a desperdiçada, evidenciando ganhos de eficiência.

Expansão Organizada da Colônia

O modelo logístico orienta quando ampliar a geração (antes do ponto de inflexão t^*), evitando tanto a falta de energia quanto investimentos prematuros. Novos módulos podem ser adicionados ao grafo de forma planejada, observando o impacto na densidade e na criticidade da rede.

Priorização de Sistemas Críticos

A prioridade operacional e a simulação de blecaute garantem que, em emergências, a energia disponível seja direcionada primeiro ao suporte à vida (Geradores, Suporte à Vida, Habitação e Logística - todos prioridade 5).

Governança Tecnológica

As decisões do sistema são baseadas em critérios transparentes e auditáveis: pesos das conexões, prioridades documentadas e algoritmos determinísticos. Qualquer recomendação (rota, corte de carga, conexão crítica) pode ser reproduzida e justificada, o que é essencial para a confiança na automação.

Redução de Desperdícios

A detecção de conexões críticas aponta onde investir em redundância para evitar perdas por interrupção; as métricas de eficiência ajudam a identificar trechos com folga ou sobrecarga, apoiando melhorias contínuas na infraestrutura.

9. Simulações Operacionais

O sistema implementa três simulações que demonstram, com dados reais da colônia, como os algoritmos e a modelagem se aplicam a situações operacionais concretas.

9.1 Distribuição de Energia com Perdas (Rota Ótima)

Simula o envio de energia do módulo gerador (MOD-EN-001) a qualquer outro módulo, calculando a rota ótima pelo Dijkstra e aplicando perdas de transmissão proporcionais à distância percorrida. O coeficiente de perda adotado é 0,0003 por metro de cabo - valor baseado em resistência ôhmica típica de cabeamento de baixa tensão.

Fórmula de eficiência por rota:

$$\text{eficiência} = \prod (1 - 0,0003 \times \text{distância_trecho})$$

O $\prod$ é usado na fórmula quando formos considerar que a corrente foi percorrida por mais de um módulo, por exemplo, do módulo MOD-EN-001 para o MOD-SV-002 para o MOD-HB-003 para o MOD-SC-004. Nesse caso, ele serve pra representar a multiplicação em sequência

Da energia restante vinda da fórmula $(1 - 0,0003 \times \text{distância_trecho}) \times (1 - 0,0003 \times \text{distância_ProximoTrecho})$.

Na tabela abaixo apenas mostramos a perda e a energia entregues devido ao comprimento do cabo que é igual a distância percorrida pela corrente

Resultados para rotas a partir de MOD-EN-001:

→ MOD-SV-002 :	75 m	perda =	2,25%	entregue =	97,75%
→ MOD-AG-007 :	120 m	perda =	3,60%	entregue =	96,40%
→ MOD-HB-003 :	155 m	perda =	4,65%	entregue =	95,35%
→ MOD-LG-006 :	210 m	perda =	6,30%	entregue =	93,70%
→ MOD-SC-004 :	215 m	perda =	6,45%	entregue =	93,55%
→ MOD-MN-005 :	220 m	perda =	6,60%	entregue =	93,40%
→ MOD-CM-008 :	230 m	perda =	6,90%	entregue =	93,10%

Perda média nas rotas ótimas: ~5,2%

Distância total de cabeamento: 930 m

Conclusão: usar a rota do Dijkstra (menor distância) reduz diretamente as perdas. Um caminho alternativo mais longo elevaria a perda proporcionalmente ao excesso de metros.

9.2 Priorização em Blecaute - Problema da Mochila

Simula uma falha de geração em que apenas uma reserva limitada de energia está disponível. O sistema precisa decidir quais módulos manter ligados para preservar ao máximo os sistemas críticos - um problema clássico de otimização combinatória análogo ao Problema da Mochila.

Estratégia: busca exaustiva por todos os $2^8 = 256$ subconjuntos possíveis de módulos. Para cada subconjunto que respeita a energia disponível, calcula-se um score lexicográfico de prioridade - primeiro maximiza módulos de prioridade 5, depois 4, e assim por diante. Isso garante que nenhum módulo de menor prioridade seja mantido em detrimento de um vital.

Exemplo - energia disponível: 250 kW

Testadas: 256 combinações (2^8)

Resultado ótimo:

✓ Ligados : MOD-EN-001 (15kW) + MOD-SV-002 (40kW)
+ MOD-HB-003 (120kW) + MOD-LG-006 (60kW)

Uso total : 235 kW de 250 kW disponíveis

✗ Desligados: MOD-SC-004, MOD-MN-005, MOD-AG-007, MOD-CM-008

→ Todos os módulos de prioridade 5 foram preservados.

→ Módulos de prioridade 4 desligados para garantir os vitais.

9.3 Análise de Falha de Conexão

Simula a remoção de uma conexão específica da rede e avalia o impacto: verifica se a rede permanece conectada, quantos módulos ficam isolados, quais rotas alternativas existem e se a conexão removida era uma ponte crítica. Permite planejar redundâncias nos pontos mais vulneráveis.

10. Smart Grid e Armazenamento de Energia

O SIGIC reproduz computacionalmente os pilares de uma rede inteligente (smart grid) - conceito exigido no item 1 do enunciado como conteúdo aplicado. Uma smart grid combina monitoramento em tempo real, roteamento inteligente de energia e resposta automática a falhas.

10.1 Armazenamento de Energia

Cada módulo possui uma capacidade de armazenamento local (baterias), representada pelo atributo capacidade (kWh). Esse valor indica por quanto tempo o módulo pode operar com sua reserva local, sem receber energia da rede.

Módulo	Consumo (kW)	Reserva (kWh)	Autonomia local
MOD-EN-001 - Geradores	15	2.000	133,3 h

MOD-LG-006 - Logística	60	80	1,3 h
MOD-SC-004 - Sensores	90	100	1,1 h
MOD-SV-002 - Suporte à Vida	40	60	1,5 h
MOD-HB-003 - Habitação	120	50	0,4 h
MOD-MN-005 - Mineração	50	40	0,8 h
MOD-AG-007 - Agricultura	50	40	0,8 h
MOD-CM-008 - Comunicação	50	40	0,8 h

Reserva total: 2.410 kWh · **Consumo total:** 475 kW

Autonomia global sem geração: ~5,1 h ($2.410 \div 475$) — *desde que o banco de baterias do gerador permaneça ativo.* Como 83% da reserva está no próprio MOD-EN-001, se o gerador falhar de fato sobram apenas ~410 kWh, ou seja, **~53 minutos**. Por isso a reserva só garante as 5 horas se houver redundância no cabeamento do gerador.

10.2 Distribuição Inteligente de Energia

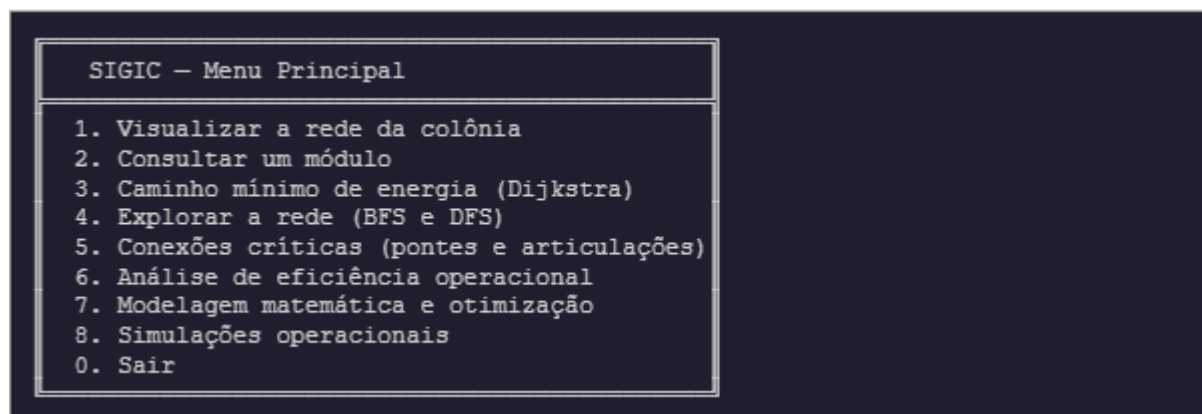
A distribuição segue os princípios de uma smart grid: a rota de energia é calculada automaticamente pelo Dijkstra (menor distância = menor perda), e em situações de emergência o sistema redireciona a energia para módulos prioritários com base no algoritmo de blecaute.

- **Monitoramento:** o status e o consumo de cada módulo são consultáveis em tempo real pelo menu (opção 2).
- **Roteamento inteligente:** o Dijkstra calcula automaticamente a rota de menor perda entre qualquer par de módulos (opção 3).
- **Corte seletivo de carga:** em blecaute, o sistema desliga módulos de menor prioridade para preservar os vitais (opção 8).
- **Resiliência:** a detecção de pontes e pontos de articulação (opção 5) identifica onde adicionar redundância para aumentar a robustez da rede.

11. Funcionalidades e Como Executar

```
# Abra o terminal na pasta do projeto e execute:
py codigo_fonte.py
# ou:
python codigo_fonte.py
```

O programa exibe o seguinte menu de navegação:



As simulações (opção 8) incluem: distribuição de energia com perdas pela rota ótima; priorização de cargas em falha de geração; e análise do impacto da falha de uma conexão. Todas as entradas são validadas, e o programa não encerra por erros de digitação.

Exemplo - Priorização em blecaute (250 kW disponíveis)

```
Energia disponível : 250 kW  
Mantidos ligados   : MOD-EN-001, MOD-SV-002, MOD-HB-003, MOD-LG-006 (uso: 235  
kW)  
Desligados         : MOD-SC-004, MOD-MN-005, MOD-AG-007, MOD-CM-008
```